INFORME FINAL

DESAFÍO II

Ude@Book

Desarrollo de la capa lógica de una red social
mediante programación orientada a objetos en C++

PRESENTADO POR

Jose Gabriel Giraldo Flórez

Juan Nicolás Rubiano Barreiro

DOCENTE

Aníbal José Guerra Soler

Medellín, Antioquia, Colombia

Octubre de 2026

Índice

1. Introducción 1

2. Análisis del problema y consideraciones 1

3. Diseño – Diagrama de clase 2

4. Estructura de datos y algoritmos esenciales 2

5. Formato de archivos 3

6. Medición del consumo de recursos 4

7. Evolución de la solución y mejoras de eficiencia 4

8. Problemas del desarrollo 5

9. Conclusiones 5

1. Introducción

Se presenta la implementación y desarrollo de Ude@Book, es la capa lógica de una red social, desarrollada en C++ con programación orientada a objetos y con interacción por consola. Donde un usuario puede iniciar sección y ver fotos de perfil de sus amigos, enviar y captar solicitudes de amistad, recorrer los álbumes de un amigo y dar like, recibir sugerencias de amistad según los amigos en común, y enviar, leer y buscar mensajes por fecha. Cada función con su respectivo informe de iteraciones, el uso de componentes externos y la memoria ocupada.

Se toma en cuenta el criterio principal de evaluación de la eficiencia, con restricciones fuertes: sin contenedores STL, sin struct ni funciones de strings, obligando el uso de estructuras de datos propias en memoria dinámica y con archivos usados exclusivamente para cargar y guardar. Teniendo como solución final el cumplimiento de las 7 funciones y el trabajo de pruebas con 1000 usuarios, generados automáticamente con ayuda de un programa externo.

Para el informe se presenta el análisis del problema y sus restricciones, el diagrama de clase, los algoritmos esenciales en alto nivel, formato de archivos y la forma de guardar los cambios, medición de los recursos, evaluación de la solución con mejoras de eficiencia medidas, problemas del desarrollo, pruebas y conclusiones. Por su parte, el repositorio contiene el código, el diagrama de clase y desarrollo completo.

2. Análisis del problema y consideraciones

El problema central es responder rápidamente preguntas sobre relaciones entre muchos usuarios: quienes son amigo, cuantos amigos tienen en común y que mensajes caen en un rango de fechas. Para lo cual el diseño se apoya en una matriz de adyacencia de bits para las amistades, listas de mensajes ordenados por fechas y usuarios ordenados por nombres.

Restricciones que condicionaron el diseño:

- **Estructuras propias:** Se usa una plantilla `Lista<T>` con memoria dinámica para reemplaza a la STL, y todo el uso de textos mediante `Lista<char>`.
- **Operaciones propias:** Recorrido de archivos, conversión entre números y texto y comparaciones de texto se programaron a mano; de la librería únicamente se leen y escriben caracteres sueltos.
- **Componentes externos:** Se apoya del uso de `<random>` para la elección de las fotos y `<chrono>` solo para la captura de la hora; teniendo en cuenta cada uno de sus usos para las métricas.
- **Archivos solo para cargar y guardar:** No se usan para procesamiento, todo procesamiento se da en memoria tal como pide el enunciado.

- No se usan conversiones de C y se toma en cuenta al menos 2 operadores sobrecargados.

3. Diseño – Diagrama de clase

Se usan nueve clases, las cuales forman un árbol de propiedad: UdeABook posee los usuarios y la red de amistades, cada usuario posee sus álbumes y sus mensajes recibidos, y cada álbum sus fotos. El Diagrama UML completo, con atributos y métodos, se entrega como archivo aparte en el repositorio.

Cada mensaje existe una sola vez: lo libera el receptor, y el emisor y la lista global de UdeABook se encarga de guardar punteros (asociaciones).

Conceptos de POO aplicados:

- Constructores de copia y `operator=` en todas las clases, se implementan con inicializaciones miembro a miembro y copia profunda donde se tiene memoria dinámica.
- Operadores sobrecargados: `operator=`, `operator[]` para las clases `Lista<T>` y `operator<` en fecha, las cuales permiten las búsquedas binarias por fecha.
- Métodos sobrecargados: Se usa la sobrecarga para tres leer (texto, numero, fecha), tres escribir, dos imprimir Ruta y dos `métricas::contar`.
- Encapsulamiento: Se toman los atributos privados y getters que devuelven referencias constantes, para evitar copiar listas.

4. Estructura de datos y algoritmos esenciales

Ninguna de las funcionalidades hace recorrido completo de todos los datos, para esto se usan accesos directos por id, búsqueda binaria por nombre y fecha, y operaciones de bits sobre la matriz de amistad.

Por su parte, `Lista<T>`. Es una lista dinámica que solo se amplía (al doble) cuando está llena. Se puede reservar capacidad exacta y eliminar, reemplazando por el último elemento.

En `RedAmistad` con la matriz de adyacencia con el uso de bits: para los 1000 usuarios, cada fila son 16 palabras de 64 bits dando un total de 128.000 bytes, dando ocho veces menos que con el uso de bool. Para saber si 2 usuarios son amigos se lee un bit; contar amigos en común es un AND entre 2 filas y contar sus bits, teniendo siempre 16 iteraciones.

I. **Cargar:** La función de cargar lee usuarios, amistades, solicitudes, mensajes, álbumes y la bitácora, en dicho orden, carácter por carácter con rutinas propias.

Como el id de usuario es una posición en usuarios.txt, cada referencia es acceso directo, y los conteos de cada línea reservan las listas con su tamaño exacto.

II. **Inicio de sección:** En esta parte el usuario se busca con una búsqueda binaria (10 comparaciones aprox.) y se compara primero la clave por la longitud. Cada amigo recibe un peso de 2,5 o 7 según sus amigos en común; además, se sortea un numero entre 0 y la suma de pesos y se recorren los pesos acumulados hasta encontrar al elegido, que se retira con su peso para evitar repetirlo. La actualización se permite una sola vez.

III. **Amistades:** Antes de permitir el enviar una solicitud se hace validaciones de que no se haga a sí mismo, que no sean amigos, que no se haya enviado de antemano y que dicha persona no nos haya enviado una. En su lugar aceptar enciende los 2 bits simétricos de la matriz.

IV. **Perfil de un amigo:** Solo se listan los álbumes públicos y de amigos. El carrusel avanza y retrocede a decisión del usuario desde consola con modulo el número de fotos, así que es circular.

V. **Sugerencias:** Se usa un OR para las filas de los amigos, lo cual da los amigos de los amigos y un AND negado que permite quitar los que ya son amigos y al propio usuario. Para cada candidato se le cuentan los amigos en común y se mantienen en un único top-K ordenado por inserción, sin ordenar los 631 candidatos.

VI. **Mensajes:** Las listas quedaron ordenadas por fecha sin ordenar nada, debido a que el archivo ya viene ordenado y los mensajes nuevos son lo más recientes. Los no leídos siempre se encuentran al final, por lo tanto, leerlos comienza desde el primerNoLeido y cuesta solo lo que está sin leer. La búsqueda lleva el rango a 00:00 y 23:59, encuentra el inicio con una búsqueda binaria y avanza hasta rebasar la fecha final: (long n + encontrados).

5. Formato de archivos

Se cargan 6 formatos de archivos de texto propios en una sola pasada, donde la primera línea representa la cantidad de registros, los campos se separan con "|", las fechas cumplen siempre el mismo formato de AAA-MM-DD:MM y los usuarios son referenciados por id.

Archivo	**Formato**
Usuarios.txt	`nombre|clave|correo|ciudad|pais|nacimiento|inscripcion|nAlbumes|nEnv`
Amistades.txt	`idA|idB`
Solicitudes.txt	`origen|destino`
Mensajes.txt	`emisor|receptor|fecha|leido|texto`

Archivo	Formato
Albumes.txt	`idUsuario|nombre|privacidad|nFotos + nFotos líneas nombre|fecha|tamano|likes`
Cambios.txt	M, L, S, A o K seguido de sus datos

Los cambios se guardan según su importancia. Mensajes enviados y lecturas (M, L) se aplican de forma inmediata en la bitácora; solicitudes, aceptaciones y likes (S, A, K), se agregan al salir de su sección del menú. Al cerrar el programa se reescriben solo los archivos marcados en la máscara cambios (1 amistades, 2 solicitudes, 4 mensajes, 8 álbumes) y se vacía la bitácora. La bitácora nunca es leída durante la sección: al cargar se reaplica, lo cual permite recuperar todas las operaciones tras cerrar el programa de forma forzada.

6. Medición del consumo de recursos

Métricas es una clase estática cuyo objetivo es contar iteraciones (cada ciclo suma sus vueltas al terminar, incluida las indirectas de lista, búsquedas y lecturas de caracteres) y usos de `<random>` y `<chrono>`. La memoria se lleva como un total que sube en cada `new` y baja para cada `delete`, más el conteo de las variables locales y temporales de la funcionalidad. Esto evita recorrer las estructuras para medirlas; mostrándose que, al destruir todo, se retorna al valor inicial.

7. Evolución de la solución y mejoras de eficiencia

Las clases se construyeron de la más simple a la más compleja, terminando con la implementación de UdeABook. En Lista, Álbum, Usuario y RedAmistad se escribió primero una versión sencilla, para después pasar a una mucho más eficiente.

Componente	Antes	Después
`Lista<T>::agregar`	Crecía de 1 en 1, copiando todo	Crece al doble solo cuando está llena
`Lista<T>::eliminar`	Desplazaba los siguientes	Reemplaza por el último
`Usuario::verificarClave`	Recorría toda la clave	Longitud primero y salida temprana
`Usuario::marcarLeidos`	Recorría todos los recibidos	Empieza en primerNoLeido
Lista global de mensajes	Reserva exacta: el primer envío copiaba 35 538 punteros	Margen de 12,5 % para la sesión
Conteo de amistades al guardar	Contaba los bits de toda la matriz	Contador en RedAmistad

Componente	Antes	Después
Carga y guardado	Listas que crecían al cargar; todo se reescribía	Reserva exacta con los conteos; solo archivos modificados

8. Problemas del desarrollo

Riesgo de liberación dos veces de un mensaje, que está en las listas del emisor y el receptor:

Para solucionarlo se implementa un único dueño (El receptor); los demás guardan punteros.

Copias superficiales de objetos con punteros (Álbum, Usuario):

Se implementa el constructor de copia y `operator=` con una copia profunda y protección contra auto asignaciones.

No uso de strings ni la conversión de números con el uso de librerías:

Textos representados con `Lista<char>` y desarrollos propios para leer, escribir y comparar.

Advertencia al quitar `Fecha::operator=` y `Lista<Fecha>::contiene` sin ==:

Se mantuvo `operator=`; las fechas se comparan solo con `operator<`.

Errores de enlazado y usuario.txt no encontrado:

Todos los .cpp implementados dentro del mismo directorio de trabajo y uso de la ruta real de la carpeta corregidos.

Datos para el cubrimiento de toda la demostración:

Se genera con semilla fija, validado con un script (orden, ids, solicitudes, no leídos al final).

9. Conclusiones

- La elección de estructuras fue lo que más pesó en la eficiencia: la matriz de bits deja la amistad en tiempo constante y los amigos en común en 16 iteraciones, y las búsquedas binarias reducen a unos 10 pasos las búsquedas por nombre y por fecha.
- Medir antes de optimizar mostró dónde estaba el costo real (crecimiento de listas, primer mensaje enviado, mensajes ya leídos) y que el 73 % de la carga es inevitable.
- Iteraciones y memoria se equilibraron de forma explícita: se aceptaron mejoras de costo mínimo en memoria y se descartaron las que la aumentaban notablemente.

- Un dueño único por objeto dinámico y la persistencia por niveles de importancia hicieron el programa seguro ante fugas y ante cierres inesperados.